

Play with GUIs Using PYTHON

The GUI (Graphical User Interface) is the simplest way to let users interact with the system. If you know how to create GUIs, you can get many more users interested. This article shows you how to create GUIs using my favourite interpreted language, Python. Since Python is platform-independent, you can use GUIs with it on Windows too.

I use Ubuntu's 10.04 32-bit desktop edition, and Python 2.6.5. We can use GUIs in Python in many ways, such as the PyZenity module, the Tkinter tool-kit and PyQt (a Nokia framework). In this article, we are going to talk about the PyZenity module and the Tkinter tool-kit.

Creating GUIs using PyZenity

This is the fastest and simplest way to create a GUI. It mixes the Zenity utility with Python. Before going directly to PyZenity, I would like to demonstrate the Zenity utility. It comes pre-installed on most systems. Just go to the terminal and type `zenity --title="Demo" --info --text="hello"` and you will see something like what's shown in Figure 1—an information box, with its title being 'Demo', and the message 'Hello'. Explore the Zenity man page for more details on Zenity.

The PyZenity module is an easy way to use the Zenity utility in Python. To use it, you have to download and compile it manually. The source is at <http://www.brianramos.com/docs/PyZenity/>. The latest version is PyZenity-0.1.5, though I'm using PyZenity-0.1.4. Follow the steps given below to install the tar.gz file:

```
tar xzf PyZenity-0.1.4.tar.gz
cd PyZenity-0.1.4
python setup.py install
```

After installing, I would recommend you first visit the documentation for this module at <http://www.brianramos.com/docs/PyZenity/>. This Web page lists all the possible GUI items you can create using this module, and their various parameters. For example, you can make info boxes, menu boxes, error dialogue boxes, progress bars, etc.

After you scan the available functions, let's try making an input box, using the 'GetText' function. Enter the following code, save it as `demo1.py`, and run it (the output is shown in Figure 2):

```
import PyZenity
a=PyZenity.GetText(text="Enter the string : ", entry_text="",
password=False)
print a
```

This will accept the 'Hello' as your input string, and will display it on the console as soon as you press <return> or the 'OK' button. It will return *None* if you press the 'Cancel' button.

Let's try and understand the program better. We call the `GetText` function following the general syntax of `module.function()`. The `text` option is the message to be shown; `entry_text` stands for the initial value of the text-box, which is an empty string in this case. The `password=False` means that you want the text to be visible to the user, not masked with asterisks. This is the beauty of this module, just 3-4 lines of programming and your GUI is ready.

While I was using this module, I had some difficulty in understanding its `List` function, because of the confusing parameters. Let's quickly take a look at it (save as `demo2.py`):

```
import PyZenity
x=PyZenity.List(["choices","test1","test2"], title="selection",
boolstyle="checkboxlist", editable=False, select_col="All", sep='|',
data=[["","hello","hi"],["","1","2"]])
print x
```

The output is as shown in Figure 3. Let's review the parameters quickly:

1. The first parameter is a tuple that names the columns of your list (here-*choices*, *test1* and *test2*).
2. The title for the dialogue box ('selection').
3. *boolstyle* : This is the *checkboxlist* or *radiolist* in the first column of the list.
4. *editable* : Offers the option of whether the list should be editable or not.
5. *select_col* : The column number whose value is to be returned if you select a row.
6. *sep* : The row separator during the return.
7. *data* : Must be in tuples, or in tuples of tuples. Fills out the columns row-wise. Since we have selected *boolstyle=checkboxlist*, the first column should be empty, otherwise it will overwrite the selection boxes. There should be an input for every column and every row.

A List dialogue box will give you an output in the form of a tuple only. You can check that out by running the above program. To explore more interfaces, read the documentation.